

FAMA AI

Fama-French 3-Factor Stock Analysis Platform

COMPLETE SOURCE CODE AUDIT & TECHNICAL REPORT

Author: Quantitative Finance & MBA Research Team

Date: June 2026

Engine Version: v1.0 (FastAPI + Statsmodels OLS + React)

■ Source File: backend/main.py

Language: PYTHON | Path: c:\Users\Sairal\Desktop\AntiGravity-All\backend\main.py

```
import os
from fastapi import FastAPI, UploadFile, File, HTTPException, Form
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import StreamingResponse, FileResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel
from typing import Dict, Any, List

from backend.utils.csv_loader import load_csv
from backend.services.regression_service import run_fama_french_regression
from backend.utils.pdf_generator import generate_pdf_report

app = FastAPI(
    title="FAMA AI - Fama-French 3-Factor Stock Analysis Platform",
    description="Backend API and web server for statistical equity factor analysis."
)

# Enable CORS for local development
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Sample stock mapping
SAMPLE_STOCKS = {
    "infosys": {
        "name": "Infosys Limited (INFY)",
        "file_name": "infosys.csv"
    },
    "tata_motors": {
        "name": "Tata Motors Limited (TATAMOTORS)",
        "file_name": "tata_motors.csv"
    }
}

class ReportRequest(BaseModel):
    stock_name: str
    results: Dict[str, Any]

@app.get("/api/stocks")
def get_stocks():
    """
    Returns the list of pre-configured sample stocks.
    """
    return [
        {"id": symbol, "name": meta["name"]}
        for symbol, meta in SAMPLE_STOCKS.items()
    ]

@app.get("/api/fama-analysis/{symbol}")
def analyze_sample_stock(symbol: str):
    """
    Loads a sample stock factor dataset, performs regression, and returns raw data + results.
    """
    if symbol not in SAMPLE_STOCKS:
        raise HTTPException(
            status_code=404,
            detail=f"Stock symbol '{symbol}' not found. Choose 'infosys' or 'tata_motors'."
        )

    stock_meta = SAMPLE_STOCKS[symbol]
    file_path = os.path.join("backend", "sample_data", stock_meta["file_name"])

    if not os.path.exists(file_path):
        raise HTTPException(
```

```

        status_code=500,
        detail=f"Sample data file {stock_meta['file_name']} is missing on the server."
    )

try:
    # Load and validate
    df = load_csv(file_path)
    # Run regression
    results = run_fama_french_regression(df)
    # Prepare raw data points for graphs
    raw_data = df.to_dict(orient="records")

    return {
        "stock_name": stock_meta["name"],
        "results": results,
        "data": raw_data
    }
except Exception as e:
    raise HTTPException(status_code=400, detail=str(e))

@app.post("/api/fama-analysis/upload")
async def analyze_uploaded_csv(file: UploadFile = File(...)):
    """
    Validates uploaded CSV file, runs Fama-French regression, and returns raw data + results.
    """
    if not file.filename.endswith('.csv'):
        raise HTTPException(status_code=400, detail="Only CSV files are accepted.")

    try:
        contents = await file.read()
        # Load and validate CSV from bytes
        df = load_csv(contents)
        # Run regression
        results = run_fama_french_regression(df)
        # Raw records
        raw_data = df.to_dict(orient="records")

        # Extrapolate name from filename
        stock_name = os.path.splitext(file.filename)[0].replace("_", " ").title() + " (Uploaded)"

        return {
            "stock_name": stock_name,
            "results": results,
            "data": raw_data
        }
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))

@app.post("/api/reports/export")
def export_pdf_report(payload: ReportRequest):
    """
    Stateless endpoint that generates and downloads the OLS report PDF based on client inputs.
    """
    try:
        pdf_buffer = generate_pdf_report(payload.stock_name, payload.results)

        # Clean filename
        safe_name = "".join(c for c in payload.stock_name if c.isalnum() or c in (' ', '-', '_')).rstrip()
        safe_name = safe_name.replace(" ", "_")
        filename = f"Fama_French_Report_{safe_name}.pdf"

        return StreamingResponse(
            pdf_buffer,
            media_type="application/pdf",
            headers={"Content-Disposition": f"attachment; filename={filename}"})
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"PDF generation failed: {str(e)}")

# Fallback serving of SPA root index.html
@app.get("/")
def serve_index():
    index_path = os.path.join("backend", "static", "index.html")
    if os.path.exists(index_path):
        return FileResponse(index_path)

```

```
        return {"message": "FAMA AI server is running. Frontend static directory is empty or setup failed."}

# Mount static files (javascript, styles, favicon)
# Must be mounted AFTER other specific endpoints to avoid route interception.
static_dir = os.path.join("backend", "static")
if os.path.exists(static_dir):
    app.mount("/", StaticFiles(directory=static_dir, html=True), name="static")
else:
    # Create directory structure for static files
    os.makedirs(os.path.join(static_dir, "js"), exist_ok=True)
    os.makedirs(os.path.join(static_dir, "css"), exist_ok=True)
    app.mount("/", StaticFiles(directory=static_dir, html=True), name="static")
```

■ Source File: backend/services/regression_service.py

Language: PYTHON | Path: c:\Users\Sairal\Desktop\AntiGravity-Allbackend\services\regression_service.py

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
from typing import Dict, Any

def run_fama_french_regression(df: pd.DataFrame) -> Dict[str, Any]:
    """
    Performs Fama-French 3-Factor OLS regression.
    Formula:  $R_{i\_Rf} = \text{Alpha} + \text{Beta1} * (R_{m\_Rf}) + \text{Beta2} * (\text{SMB}) + \text{Beta3} * (\text{HML})$ 

    Parameters:
        df: pd.DataFrame with Date, Ri_Rf, Rm_Rf, SMB, HML

    Returns:
        Dict containing OLS summary statistics, coefficients, p-values, t-stats,
        R-squared, confidence intervals, and qualitative interpretations.
    """
    # Define variables
    y = df['Ri_Rf']
    X = df[['Rm_Rf', 'SMB', 'HML']]

    # Add constant for Alpha (intercept)
    X_with_const = sm.add_constant(X)

    # Run OLS regression
    model = sm.OLS(y, X_with_const)
    results = model.fit()

    # Extract coefficients
    params = results.params
    pvalues = results.pvalues
    tvalues = results.tvalues
    conf_int = results.conf_int()
    bse = results.bse # Standard errors

    # Output structure
    regression_stats = {
        "num_observations": int(results.nobs),
        "r_squared": float(results.rsquared),
        "adj_r_squared": float(results.rsquared_adj),
        "f_statistic": float(results.fvalue),
        "f_pvalue": float(results.f_pvalue),
        "factors": {
            "Alpha": {
                "coefficient": float(params['const']),
                "p_value": float(pvalues['const']),
                "t_stat": float(tvalues['const']),
                "std_err": float(bse['const']),
                "conf_lower": float(conf_int.loc['const', 0]),
                "conf_upper": float(conf_int.loc['const', 1]),
                "significant": bool(pvalues['const'] < 0.05)
            },
            "Market": {
                "coefficient": float(params['Rm_Rf']),
                "p_value": float(pvalues['Rm_Rf']),
                "t_stat": float(tvalues['Rm_Rf']),
                "std_err": float(bse['Rm_Rf']),
                "conf_lower": float(conf_int.loc['Rm_Rf', 0]),
                "conf_upper": float(conf_int.loc['Rm_Rf', 1]),
                "significant": bool(pvalues['Rm_Rf'] < 0.05)
            },
            "SMB": {
                "coefficient": float(params['SMB']),
                "p_value": float(pvalues['SMB']),
                "t_stat": float(tvalues['SMB']),
                "std_err": float(bse['SMB']),
                "conf_lower": float(conf_int.loc['SMB', 0]),
                "conf_upper": float(conf_int.loc['SMB', 1]),
            }
        }
    }
```

```

        "significant": bool(pvalues['SMB'] < 0.05)
    },
    "HML": {
        "coefficient": float(params['HML']),
        "p_value": float(pvalues['HML']),
        "t_stat": float(tvalues['HML']),
        "std_err": float(bse['HML']),
        "conf_lower": float(conf_int.loc['HML', 0]),
        "conf_upper": float(conf_int.loc['HML', 1]),
        "significant": bool(pvalues['HML'] < 0.05)
    }
}

# Generate qualitative interpretations
regression_stats["interpretations"] = generate_interpretations(regression_stats)

return regression_stats

def generate_interpretations(stats: Dict[str, Any]) -> Dict[str, str]:
    """
    Generates plain-English financial interpretations of regression metrics.
    """
    factors = stats["factors"]
    r2 = stats["r_squared"]

    mkt = factors["Market"]
    smb = factors["SMB"]
    hml = factors["HML"]
    alpha = factors["Alpha"]

    interpretations = {}

    # 1. Market Beta Interpretation
    mkt_coeff = mkt["coefficient"]
    mkt_sig = " (statistically significant)" if mkt["significant"] else " (not statistically significant)"
    if mkt_coeff > 1.1:
        interpretations["market"] = f"This stock is highly sensitive to market movements (Beta = {mkt_coeff:.2f}{mkt_sig}). It tends to outperform the market."
    elif mkt_coeff < 0.9:
        interpretations["market"] = f"This stock is relatively stable compared to the market (Beta = {mkt_coeff:.2f}{mkt_sig}). It tends to underperform the market."
    else:
        interpretations["market"] = f"This stock moves in sync with the broad market index (Beta = {mkt_coeff:.2f}{mkt_sig}). It has average market sensitivity."

    # 2. SMB (Size Factor) Interpretation
    smb_coeff = smb["coefficient"]
    smb_sig = " (statistically significant)" if smb["significant"] else " (not statistically significant)"
    if smb_coeff > 0.15:
        interpretations["smb"] = f"The stock shows characteristics associated with smaller companies (SMB Beta = {smb_coeff:.2f}{smb_sig}). It tends to outperform large-cap companies."
    elif smb_coeff < -0.15:
        interpretations["smb"] = f"The stock behaves more like large-cap companies (SMB Beta = {smb_coeff:.2f}{smb_sig}). It tends to underperform smaller companies."
    else:
        interpretations["smb"] = f"The stock is size-neutral (SMB Beta = {smb_coeff:.2f}{smb_sig}). Its performance is not significantly related to company size."

    # 3. HML (Value vs Growth Factor) Interpretation
    hml_coeff = hml["coefficient"]
    hml_sig = " (statistically significant)" if hml["significant"] else " (not statistically significant)"
    if hml_coeff > 0.15:
        interpretations["hml"] = f"The stock exhibits value-oriented characteristics (HML Beta = {hml_coeff:.2f}{hml_sig}). It has a higher value-to-growth ratio."
    elif hml_coeff < -0.15:
        interpretations["hml"] = f"The stock exhibits growth-oriented characteristics (HML Beta = {hml_coeff:.2f}{hml_sig}). It tends to outperform value stocks."
    else:
        interpretations["hml"] = f"The stock behaves like a blend of value and growth (HML Beta = {hml_coeff:.2f}{hml_sig}). It has balanced characteristics."

    # 4. Alpha (Abnormal Return) Interpretation
    alpha_coeff = alpha["coefficient"] * 100 # Convert to percent
    alpha_sig = " (statistically significant)" if alpha["significant"] else " (not statistically significant)"
    if alpha["significant"] and alpha_coeff > 0:
        interpretations["alpha"] = f"The stock generated a positive abnormal return of {alpha_coeff:.3f}% per period (Alpha = {alpha_coeff:.4f}{alpha_sig})."
    elif alpha["significant"] and alpha_coeff < 0:
        interpretations["alpha"] = f"The stock underperformed by {abs(alpha_coeff):.3f}% per period (Alpha = {alpha_coeff:.4f}{alpha_sig})."
    else:
        interpretations["alpha"] = f"The stock's abnormal return (Alpha = {alpha_coeff:.4f}{alpha_sig}) is statistically insignificant."

    # 5. R-squared Interpretation
    r2_pct = r2 * 100

```

```
if r2 > 0.85:
    interpretations["r_squared"] = f"The Fama-French 3-Factor model explains a very large portion ({r2_pct:.1f}%) of the stock's
elif r2 >= 0.50:
    interpretations["r_squared"] = f"The Fama-French 3-Factor model explains a moderate portion ({r2_pct:.1f}%) of the returns.
else:
    interpretations["r_squared"] = f"The Fama-French 3-Factor model explains a small portion ({r2_pct:.1f}%) of the returns. The

return interpretations
```

■ Source File: backend/utils/csv_loader.py

Language: PYTHON | Path: c:\Users\Sairal\Desktop\AntiGravity-All\backend\utils\csv_loader.py

```
import pandas as pd
import io
from typing import Union

REQUIRED_COLUMNS = ["Date", "Ri_Rf", "Rm_Rf", "SMB", "HML"]

def load_csv(file_source: Union[str, bytes, io.BytesIO]) -> pd.DataFrame:
    """
    Loads and validates a Fama-French CSV dataset.

    Parameters:
        file_source: Can be a file path (str), raw bytes (from file upload), or a BytesIO stream.

    Returns:
        pd.DataFrame containing columns: Date, Ri_Rf, Rm_Rf, SMB, HML

    Raises:
        ValueError: If required columns are missing, data is empty, or numeric columns contain non-numeric data.
    """
    try:
        if isinstance(file_source, bytes):
            df = pd.read_csv(io.BytesIO(file_source))
        elif isinstance(file_source, io.BytesIO):
            df = pd.read_csv(file_source)
        else:
            # Assume it's a file path string
            df = pd.read_csv(file_source)
    except Exception as e:
        raise ValueError(f"Failed to read CSV file: {str(e)}")

    if df.empty:
        raise ValueError("The uploaded CSV file is empty.")

    # Clean up column names (strip whitespace and handle casing)
    df.columns = [col.strip() for col in df.columns]

    # Case-insensitive column matching
    column_mapping = {}
    lower_required = [col.lower() for col in REQUIRED_COLUMNS]

    for col in df.columns:
        if col.lower() in lower_required:
            # Map the actual column name to the standard capitalized name
            idx = lower_required.index(col.lower())
            column_mapping[col] = REQUIRED_COLUMNS[idx]

    # Check if all required columns are found
    missing_cols = [col for col in REQUIRED_COLUMNS if col not in column_mapping.values()]
    if missing_cols:
        raise ValueError(f"Missing required columns: {', '.join(missing_cols)}. The CSV must contain Date, Ri_Rf, Rm_Rf, SMB, HML.")

    # Rename columns to our standard format
    df = df.rename(columns=column_mapping)

    # Select only the required columns
    df = df[REQUIRED_COLUMNS]

    # Drop rows where all elements are NaN
    df = df.dropna(how='all')

    # Check if we have at least 3 rows to run OLS regression
    if len(df) < 4:
        raise ValueError("At least 4 data points are required to run the Fama-French 3-Factor regression.")

    # Validate Date column
    df['Date'] = df['Date'].astype(str).str.strip()
    if df['Date'].isnull().any() or (df['Date'] == '').any():
        raise ValueError("The 'Date' column contains empty or null values.")
```



```
# Validate numeric columns
numeric_cols = ["Ri_Rf", "Rm_Rf", "SMB", "HML"]
for col in numeric_cols:
    try:
        df[col] = pd.to_numeric(df[col])
    except Exception:
        raise ValueError(f"Column '{col}' must contain only numeric values.")

    # Drop rows where numeric columns are NaN
    df = df.dropna(subset=[col])

if len(df) < 4:
    raise ValueError("Not enough valid numeric rows to perform regression analysis.")

return df
```

■ Source File: backend/utils/pdf_generator.py

Language: PYTHON | Path: c:\Users\Sairal\Desktop\AntiGravity-All\backend\utils\pdf_generator.py

```
import io
from reportlab.lib.pagesizes import letter
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib import colors
from typing import Dict, Any

def generate_pdf_report(stock_name: str, results: Dict[str, Any]) -> io.BytesIO:
    """
    Generates a structured, presentation-ready PDF report of the OLS Fama-French analysis.

    Parameters:
        stock_name: Name of the stock analyzed.
        results: Dict from run_fama_french_regression.

    Returns:
        BytesIO stream of the PDF file.
    """
    buffer = io.BytesIO()

    # Page setup
    doc = SimpleDocTemplate(
        buffer,
        pagesize=letter,
        rightMargin=40,
        leftMargin=40,
        topMargin=40,
        bottomMargin=40
    )

    styles = getSampleStyleSheet()

    # Custom styles
    title_style = ParagraphStyle(
        'DocTitle',
        parent=styles['Heading1'],
        fontName='Helvetica-Bold',
        fontSize=24,
        textColor=colors.HexColor('#0F172A'), # Slate 900
        spaceAfter=12
    )

    subtitle_style = ParagraphStyle(
        'DocSubtitle',
        parent=styles['Normal'],
        fontName='Helvetica',
        fontSize=11,
        textColor=colors.HexColor('#64748B'), # Slate 500
        spaceAfter=20
    )

    heading2_style = ParagraphStyle(
        'SectionHeading',
        parent=styles['Heading2'],
        fontName='Helvetica-Bold',
        fontSize=15,
        textColor=colors.HexColor('#1E3A8A'), # Navy blue
        spaceBefore=14,
        spaceAfter=8,
        keepWithNext=True
    )

    body_style = ParagraphStyle(
        'BodyTextCustom',
        parent=styles['BodyText'],
        fontName='Helvetica',
        fontSize=10,
        textColor=colors.HexColor('#334155'), # Slate 700
    )
```

```

        leading=14
    )

bullet_style = ParagraphStyle(
    'BulletCustom',
    parent=styles['Normal'],
    fontName='Helvetica',
    fontSize=9.5,
    textColor=colors.HexColor('#1E293B'), # Slate 800
    leftIndent=15,
    firstLineIndent=-10,
    spaceAfter=6,
    leading=13
)

formula_style = ParagraphStyle(
    'FormulaText',
    parent=styles['Normal'],
    fontName='Courier-Bold',
    fontSize=10,
    textColor=colors.HexColor('#312E81'), # Indigo 900
    alignment=1, # Centered
    backColor=colors.HexColor('#EEF2F6'),
    borderPadding=8,
    spaceAfter=15
)

elements = []

# Header Section
elements.append(Paragraph("FAMA AI", title_style))
elements.append(Paragraph(f"Fama-French 3-Factor Regression Report &bull; Academic Analysis &bull; {stock_name}", subtitle_style))

# Divider line
divider = Table([[""]], colWidths=[532], rowHeights=[2])
divider.setStyle(TableStyle([
    ('BACKGROUND', (0,0), (-1,-1), colors.HexColor('#1E3A8A')),
    ('TOPPADDING', (0,0), (-1,-1), 0),
    ('BOTTPADDING', (0,0), (-1,-1), 0),
]))
elements.append(divider)
elements.append(Spacer(1, 15))

# Executive Summary / Regression Equation
elements.append(Paragraph("Regression Equation", heading2_style))
equation_text = (
    "R<sub>i</sub> - R<sub>f</sub> = &alpha; + &beta;<sub>Market</sub>(R<sub>m</sub> - R<sub>f</sub>) + "
    "s &times; SMB + h &times; HML + &epsilon;"
)
elements.append(Paragraph(equation_text, formula_style))

# Stock Overview Description
intro_p = (
    f"This academic report presents the OLS regression analysis results for <b>{stock_name}</b> "
    f"using the Fama-French 3-Factor Model. The analysis calculates exposure to systemic risk factors "
    f"over {results['num_observations']} observation periods. Below are the coefficients, p-values, "
    f"statistical significance, and economic interpretations."
)
elements.append(Paragraph(intro_p, body_style))
elements.append(Spacer(1, 12))

# Regression Results Table
elements.append(Paragraph("Model Coefficients & Statistics", heading2_style))

factors = results["factors"]

# Prepare data for table
table_data = [
    ["Factor", "Coefficient", "Std Error", "t-Statistic", "p-Value", "95% Conf. Interval", "Significant"]
]

for factor_name, data in factors.items():
    # Map standard keys
    display_name = {
        "Alpha": "Alpha (Intercept)",

```

```

        "Market": "Market Premium (Rm-Rf)",
        "SMB": "Size Factor (SMB)",
        "HML": "Style Factor (HML)"
    }.get(factor_name, factor_name)

    conf_interval = f"[{data['conf_lower']:.4f}, {data['conf_upper']:.4f}]"
    sig_text = "Yes (p < 0.05)" if data["significant"] else "No"

    table_data.append([
        display_name,
        f"{data['coefficient']:.4f}",
        f"{data['std_err']:.4f}",
        f"{data['t_stat']:.2f}",
        f"{data['p_value']:.4f}",
        conf_interval,
        sig_text
    ])

# Table layout & styles
col_widths = [135, 65, 55, 60, 50, 110, 57]
stats_table = Table(table_data, colWidths=col_widths)
stats_table.setStyle(TableStyle([
    ('BACKGROUND', (0,0), (-1,0), colors.HexColor('#0F172A')), # Dark navy header
    ('TEXTCOLOR', (0,0), (-1,0), colors.whitesmoke),
    ('ALIGN', (0,0), (-1,-1), 'CENTER'),
    ('ALIGN', (0,1), (0,-1), 'LEFT'), # Left align first column
    ('FONTNAME', (0,0), (-1,0), 'Helvetica-Bold'),
    ('FONTSIZE', (0,0), (-1,-1), 8),
    ('BOTTOMPADDING', (0,0), (-1,-1), 6),
    ('TOPPADDING', (0,0), (-1,-1), 6),
    ('GRID', (0,0), (-1,-1), 0.5, colors.HexColor('#CBD5E1')), # Light grey border
    ('ROWBACKGROUNDS', (0,1), (-1,-1), [colors.white, colors.HexColor('#F8FAFC')]), # Alternating rows
]))
elements.append(stats_table)
elements.append(Spacer(1, 10))

# Model Diagnostics Table
elements.append(Paragraph("Model Fit Diagnostics", heading2_style))
diagnostics_data = [
    [
        Paragraph("<b>Number of Observations:</b>", body_style), f"{results['num_observations']}",
        Paragraph("<b>R-Squared (R<sup>2</sup>):</b>", body_style), f"{results['r_squared']:.4f}"
    ],
    [
        Paragraph("<b>F-Statistic:</b>", body_style), f"{results['f_statistic']:.2f}",
        Paragraph("<b>Adjusted R-Squared:</b>", body_style), f"{results['adj_r_squared']:.4f}"
    ],
    [
        Paragraph("<b>Prob (F-statistic):</b>", body_style), f"{results['f_pvalue']:.4e}",
        "", ""
    ]
]
diag_table = Table(diagnostics_data, colWidths=[150, 100, 150, 132])
diag_table.setStyle(TableStyle([
    ('VALIGN', (0,0), (-1,-1), 'MIDDLE'),
    ('BOTTOMPADDING', (0,0), (-1,-1), 4),
    ('TOPPADDING', (0,0), (-1,-1), 4),
    ('LINEBELOW', (0,0), (-1,-1), 0.5, colors.HexColor('#E2E8F0')),
    ('SPAN', (2,2), (3,2)), # Span empty cells
]))
elements.append(diag_table)
elements.append(Spacer(1, 15))

# Qualitative Interpretations
elements.append(Paragraph("Fama-French Factor Interpretation", heading2_style))

interpretations = results["interpretations"]

bullets = [
    f"<b>Market Sensitivity (&beta;):</b> {interpretations['market']}",
    f"<b>Size Premium Exposure (SMB):</b> {interpretations['smb']}",
    f"<b>Style Premium Exposure (HML):</b> {interpretations['hml']}",
    f"<b>Abnormal Return (&alpha;):</b> {interpretations['alpha']}",
    f"<b>Model Explanatory Power (R<sup>2</sup>):</b> {interpretations['r_squared']}"
]

```

```
for bullet in bullets:
    elements.append(Paragraph(f"&bull; {bullet}", bullet_style))

elements.append(Spacer(1, 20))

# Academic Disclaimer Footer
disclaimer_style = ParagraphStyle(
    'Disclaimer',
    parent=styles['Normal'],
    fontName='Helvetica-Oblique',
    fontSize=8,
    textColor=colors.HexColor('#94A3B8'),
    alignment=1, # Centered
    spaceBefore=20
)
elements.append(Paragraph(
    "FAMA AI is an academic analysis project. This analysis is based on historical factor data and "
    "should not be construed as investment advice. Past performance is not indicative of future results.",
    disclaimer_style
))

# Build Document
doc.build(elements)
buffer.seek(0)
return buffer
```

■ Source File: backend/static/index.html

Language: HTML | Path: c:\Users\Saira\Desktop\AntiGravity-AI\backend\static\index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>FAMA AI - Fama-French 3-Factor Stock Analysis Platform</title>

  <!-- Global Error Handler to Diagnose Startup Crashes -->
  <script>
    window.addEventListener('error', function(e) {
      var root = document.getElementById('root');
      if (root) {
        root.innerHTML = `
          <div class="min-h-screen flex flex-col items-center justify-center bg-slate-950 p-6 font-sans">
            <div class="max-w-xl w-full bg-rose-950/20 border border-rose-500/30 p-6 rounded-xl text-rose-200">
              <h3 class="text-lg font-bold mb-2 text-rose-400">■ Front-End Runtime Error</h3>
              <p class="text-sm font-semibold mb-1">Message: ${e.message}</p>
              <p class="text-xs font-mono text-gray-400">File: ${e.filename} (Line ${e.lineno}, Col ${e.colno})</p>
              <p class="text-xs text-gray-500 mt-4 leading-relaxed">Please send this error report back to Antigravity so we can resolve this</p>
            </div>
          </div>
        `;
      }
    });
  </script>

  <!-- SEO Meta Tags -->
  <meta name="description" content="An academic equity research tool that automates Fama-French 3-Factor OLS regression analysis on Nifty 50 stocks.">
  <meta name="keywords" content="Fama-French, Stock Analysis, OLS Regression, NIFTY 50, Finance, MBA Project, BTech Project">

  <!-- Tailwind CSS CDN -->
  <script src="https://cdn.tailwindcss.com"></script>

  <!-- Tailwind Configuration for Slate overrides and custom styles -->
  <script>
    tailwind.config = {
      theme: {
        extend: {
          colors: {
            slate: {
              950: '#030712',
              900: '#0f172a',
              850: '#111827',
              800: '#1e293b',
              700: '#334155',
            }
          }
        }
      }
    }
  </script>

  <!-- Custom Stylesheet -->
  <link rel="stylesheet" href="/css/styles.css">

  <!-- React & React DOM CDNs -->
  <script crossorigin src="https://unpkg.com/react@18.2.0/umd/react.production.min.js"></script>
  <script crossorigin src="https://unpkg.com/react-dom@18.2.0/umd/react-dom.production.min.js"></script>

  <!-- Plotly.js CDN -->
  <script src="https://cdn.plot.ly/plotly-2.24.1.min.js" charset="utf-8"></script>
</head>
<body class="bg-slate-950 text-gray-100 min-h-screen">

  <!-- React App Mount Point -->
  <div id="root">
    <!-- Server-Side Pre-loader for Premium First Impression -->
    <div class="min-h-screen flex flex-col items-center justify-center bg-slate-950 text-gray-100">
```

```
<div class="flex flex-col items-center gap-4 animate-pulse">
  <div class="w-12 h-12 rounded-xl bg-gradient-to-tr from-indigo-500 to-cyan-400 flex items-center justify-center shadow-lg sh
    <svg class="w-7 h-7 text-slate-900" fill="none" viewBox="0 0 24 24" stroke="currentColor" stroke-width="2.5">
      <path stroke-linecap="round" stroke-linejoin="round" d="M12 6.25v13m0-13C10.832 5.477 9.246 5 7.5 5S4.168 5.477 3 6.25"
    </svg>
  </div>
  <div class="text-center">
    <h2 class="text-lg font-bold tracking-wider text-white uppercase font-sans">FAMA AI</h2>
    <p class="text-xs text-gray-500 font-mono mt-1">Bootstrapping Client Components...</p>
  </div>
</div>
</div>
</div>

<!-- Main Modular Application Entrypoint -->
<script type="module" src="/js/app.js"></script>

</body>
</html>
```

■ Source File: backend/static/css/styles.css

Language: CSS | Path: c:\Users\Saira\Desktop\AntiGravity-All\backend\static\css\styles.css

```
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&family=Outfit:wght@400;500;600;700;800&family=');

/* HTML scroll behavior for smooth transition from hero to dashboard */
html {
  scroll-behavior: smooth;
}

/* Main Body Configuration */
body {
  font-family: 'Inter', sans-serif;
  background-color: #030712; /* Slate-950 equivalent */
  color: #f3f4f6; /* Gray-100 equivalent */
  overflow-x: hidden;
}

/* Headings font */
h1, h2, h3, h4, h5, h6 {
  font-family: 'Outfit', sans-serif;
}

/* Monospace stats/math font */
.mono-font {
  font-family: 'Fira Code', monospace;
}

/* Custom Sleek Scrollbar */
::-webkit-scrollbar {
  width: 6px;
  height: 6px;
}

::-webkit-scrollbar-track {
  background: #0b0f19;
}

::-webkit-scrollbar-thumb {
  background: #1f2937;
  border-radius: 3px;
}

::-webkit-scrollbar-thumb:hover {
  background: #374151;
}

/* Glassmorphism Styles */
.glass-panel {
  background: rgba(17, 24, 39, 0.7);
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  border: 1px solid rgba(255, 255, 255, 0.06);
  border-radius: 1rem;
}

.glass-panel-hover {
  transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1);
}

.glass-panel-hover:hover {
  transform: translateY(-2px);
  background: rgba(17, 24, 39, 0.85);
  border-color: rgba(99, 102, 241, 0.3) !important; /* Indigo glow on hover */
  box-shadow: 0 10px 25px -5px rgba(0, 0, 0, 0.4),
    0 0 20px 0 rgba(99, 102, 241, 0.08);
}

/* Glowing Neon Elements */
.glow-text-indigo {
  text-shadow: 0 0 15px rgba(99, 102, 241, 0.5);
}
```



```

}

.glow-text-emerald {
  text-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
}

.glow-text-cyan {
  text-shadow: 0 0 15px rgba(6, 182, 212, 0.5);
}

/* Gradient Borders */
.gradient-border-indigo {
  position: relative;
  border-radius: 1rem;
  background: linear-gradient(to right, #6366f1, #3b82f6);
  padding: 1px;
}

.gradient-border-content {
  background: #0f172a;
  border-radius: 15px;
}

/* Glowing card borders for active selected targets */
.glow-border-indigo {
  box-shadow: 0 0 20px 1px rgba(99, 102, 241, 0.15);
  border-color: rgba(99, 102, 241, 0.35) !important;
}

/* Emerald Pulsing Button Glow */
.pulse-glow-emerald {
  animation: pulseEmerald 2s infinite;
}

@keyframes pulseEmerald {
  0% {
    box-shadow: 0 0 0 0 rgba(16, 185, 129, 0.4);
  }
  70% {
    box-shadow: 0 0 0 10px rgba(16, 185, 129, 0);
  }
  100% {
    box-shadow: 0 0 0 0 rgba(16, 185, 129, 0);
  }
}

/* Gradient Text Highlights */
.gradient-text {
  background: linear-gradient(135deg, #a5b4fc 0%, #6366f1 50%, #22d3ee 100%);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
}

/* Subtle Card Gradient Overlays */
.card-gradient-overlay {
  background: linear-gradient(180deg, rgba(255, 255, 255, 0.015) 0%, rgba(255, 255, 255, 0) 100%);
}

/* Blinking Typewriter Cursor */
.cursor-blink {
  animation: cursorBlink 0.8s infinite;
}

@keyframes cursorBlink {
  0%, 100% { opacity: 0; }
  50% { opacity: 1; }
}

/* Radial Glow Background Orb for Hero Landing */
.bg-gradient-orb {
  position: absolute;
  width: 650px;
  height: 650px;
  border-radius: 50%;
  background: radial-gradient(circle, rgba(99, 102, 241, 0.08) 0%, rgba(34, 211, 238, 0.03) 50%, rgba(0,0,0,0) 100%);
}

```

```

    filter: blur(80px);
    z-index: 0;
    top: 40%;
    left: 50%;
    transform: translate(-50%, -50%);
    pointer-events: none;
}

/* Floating animation for Scroll Chevron */
.animate-bounce-slow {
  animation: bounceSlow 2s infinite;
}

@keyframes bounceSlow {
  0%, 100% {
    transform: translateY(0);
  }
  50% {
    transform: translateY(-8px);
  }
}

/* Animations */
.animate-fade-in {
  animation: fadeIn 0.4s ease-out forwards;
}

.animate-slide-up {
  animation: slideUp 0.5s cubic-bezier(0.16, 1, 0.3, 1) forwards;
}

@keyframes fadeIn {
  from {
    opacity: 0;
  }
  to {
    opacity: 1;
  }
}

@keyframes slideUp {
  from {
    opacity: 0;
    transform: translateY(16px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

/* Table styles */
.styled-table th {
  font-family: 'Outfit', sans-serif;
  letter-spacing: 0.05em;
}

/* Plotly customization overrides */
.js-plotly-plot .plotly .modebar {
  background-color: transparent !important;
}

.js-plotly-plot .plotly .user-select-none {
  cursor: default !important;
}

/* Plotly range slider styling override */
.js-plotly-plot .plotly .slider-container {
  font-size: 10px !important;
}

/* Animations for Dashboard Name Title Sequence */
.letter-zoom-fade {
  animation: letterZoomFade 0.2s cubic-bezier(0.34, 1.56, 0.64, 1) forwards;
  display: inline-block;
}

```

```

}

@keyframes letterZoomFade {
  0% {
    opacity: 0;
    transform: scale(0.3) rotate(-15deg);
    filter: blur(8px);
  }
  50% {
    opacity: 0.8;
    transform: scale(1.15) rotate(5deg);
    filter: blur(2px);
  }
  100% {
    opacity: 1;
    transform: scale(1) rotate(0deg);
    filter: blur(0);
  }
}

.word-glow-in {
  animation: wordGlowIn 1.8s cubic-bezier(0.25, 1, 0.5, 1) forwards;
  display: inline-block;
  background: linear-gradient(135deg, #a5b4fc 0%, #6366f1 50%, #22d3ee 100%);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
}

@keyframes wordGlowIn {
  0% {
    opacity: 0;
    transform: scale(0.85);
    filter: blur(12px);
    text-shadow: 0 0 0px rgba(99, 102, 241, 0);
  }
  12% {
    opacity: 1;
    transform: scale(1.02);
    filter: blur(0);
    text-shadow: 0 0 15px rgba(99, 102, 241, 0.4), 0 0 30px rgba(34, 211, 238, 0.2);
  }
  15%, 85% {
    opacity: 1;
    transform: scale(1);
    filter: blur(0);
    text-shadow: 0 0 30px rgba(99, 102, 241, 0.7), 0 0 60px rgba(34, 211, 238, 0.4), 0 0 90px rgba(99, 102, 241, 0.2);
  }
  95% {
    opacity: 0.2;
    transform: scale(1.03);
    filter: blur(4px);
    text-shadow: 0 0 15px rgba(99, 102, 241, 0.2);
  }
  100% {
    opacity: 0;
    transform: scale(1.05);
    filter: blur(15px);
    text-shadow: 0 0 0px rgba(99, 102, 241, 0);
  }
}

/* Make sure the container holds the height during animation to avoid layout shifts */
.title-container-fixed {
  min-height: 120px;
  display: flex;
  align-items: center;
  justify-content: center;
}

```

■ Source File: backend/static/js/app.js

Language: JAVASCRIPT | Path: c:\Users\Sairal\Desktop\AntiGravity-Allbackend\static\js\app.js

```
// =====
// IMPORTS (CDN STYLE SAFE)
// =====
import {
  Header,
  QuickGuide,
  HeroSection,
  StockSelector,
  UploadComponent,
  ResultsTable,
  ModelDiagnostics,
  InterpretationPanel,
  ExportReportButton
} from "./components.js";

const React = window.React;
const ReactDOM = window.ReactDOM;
const Plotly = window.Plotly;
const {
  useState,
  useEffect,
  useRef
} = React;

// =====
// CHART 1 - EXCESS RETURN TREND
// =====
function ExcessReturnTrendChart({ rawData }) {
  const chartRef = useRef(null);

  useEffect(() => {
    if (!chartRef.current || !rawData || rawData.length === 0) return;

    const dates = rawData.map(d => d.Date);
    const riRf = rawData.map(d => d.Ri_Rf);
    const rmRf = rawData.map(d => d.Rm_Rf);

    const traces = [
      {
        x: dates,
        y: riRf,
        name: "Stock Excess Return",
        type: "scatter",
        mode: "lines+markers",
        line: { color: "#6366f1", width: 2.5 },
        marker: { size: 5, color: "#818cf8" },
        fill: "tozeroy",
        fillcolor: "rgba(99, 102, 241, 0.05)"
      },
      {
        x: dates,
        y: rmRf,
        name: "Market Excess Return",
        type: "scatter",
        mode: "lines",
        line: { color: "#06b6d4", width: 1.5, dash: "dash" }
      }
    ];

    Plotly.newPlot(chartRef.current, traces, {
      paper_bgcolor: "transparent",
      plot_bgcolor: "transparent",
      margin: { t: 20, l: 40, r: 20, b: 40 },
      font: { color: "#94a3b8", family: "Inter, sans-serif" },
      hovermode: "x unified",
      xaxis: {
        gridcolor: "#1e293b",

```

```

        tickcolor: "#334155",
        zeroline: false,
        rangeslider: { visible: true },
        type: "date"
    },
    yaxis: {
        gridcolor: "#1e293b",
        tickcolor: "#334155",
        zerolinecolor: "#475569"
    },
    legend: {
        orientation: "h",
        yanchor: "bottom",
        y: 1.02,
        xanchor: "right",
        x: 1
    }
}, { responsive: true, displayModeBar: false });
}, [rawData]);

return React.createElement("div", { ref: chartRef, className: "w-full h-80" });
}

// =====
// CHART 2 - FACTOR EXPOSURE
// =====
function FactorExposureChart({ factors }) {
    const chartRef = useRef(null);

    useEffect(() => {
        if (!chartRef.current || !factors) return;

        const values = [
            factors.Market.coefficient,
            factors.SMB.coefficient,
            factors.HML.coefficient
        ];

        Plotly.newPlot(chartRef.current, [{
            x: ["Market Sensitivity (Rm-Rf)", "Size Sensitivity (SMB)", "Style Sensitivity (HML)"],
            y: values,
            type: "bar",
            marker: {
                color: ["#6366f1", "#06b6d4", "#10b981"],
                line: { color: "rgba(255,255,255,0.05)", width: 1 }
            },
            text: values.map(v => v.toFixed(3)),
            textposition: "outside",
            textfont: { color: "#f3f4f6", family: "Fira Code, monospace" }
        }], {
            paper_bgcolor: "transparent",
            plot_bgcolor: "transparent",
            font: { color: "#94a3b8", family: "Inter, sans-serif" },
            margin: { t: 25, l: 40, r: 20, b: 30 },
            xaxis: { gridcolor: "transparent" },
            yaxis: { gridcolor: "#1e293b", zerolinecolor: "#475569" }
        }, { responsive: true, displayModeBar: false });
    }, [factors]);

    return React.createElement("div", { ref: chartRef, className: "w-full h-80" });
}

// =====
// CHART 3 - P VALUE CHART
// =====
function PValueComparisonChart({ factors }) {
    const chartRef = useRef(null);

    useEffect(() => {
        if (!chartRef.current || !factors) return;

        const pvals = [

```

```

    factors.Market.p_value,
    factors.SMB.p_value,
    factors.HML.p_value
  ];

  const colors = pvals.map(p => p < 0.05 ? "#10b981" : "#64748b");

  Plotly.newPlot(chartRef.current, [{
    x: ["Market (Rm-Rf)", "Size (SMB)", "Style (HML)"],
    y: pvals,
    type: "bar",
    marker: { color: colors },
    text: pvals.map(p => p.toFixed(4)),
    textposition: "outside",
    textfont: { color: "#f3f4f6", family: "Fira Code, monospace" }
  ]), {
    paper_bgcolor: "transparent",
    plot_bgcolor: "transparent",
    font: { color: "#94a3b8", family: "Inter, sans-serif" },
    margin: { t: 30, l: 40, r: 20, b: 30 },
    xaxis: { gridcolor: "transparent" },
    yaxis: {
      gridcolor: "#1e293b",
      zerolinecolor: "#475569",
      range: [0, Math.max(0.1, ...pvals) * 1.15]
    },
    shapes: [{
      type: "line",
      x0: 0,
      x1: 1,
      y0: 0.05,
      y1: 0.05,
      xref: "paper",
      yref: "y",
      line: { color: "#ef4444", width: 1.5, dash: "dash" }
    }]
  }, { responsive: true, displayModeBar: false });

  }, [factors]);

  return React.createElement("div", { ref: chartRef, className: "w-full h-80" });
}

// =====
// MAIN APP
// =====
function App() {

  const [stocks, setStocks] = useState([]);
  const [selectedStock, setSelectedStock] = useState("");
  const [currentAnalysis, setCurrentAnalysis] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  const [showUpload, setShowUpload] = useState(false);

  // Load stocks
  useEffect(() => {
    fetch("/api/stocks")
      .then(res => res.json())
      .then(data => {
        setStocks(data);
        if (data.length > 0) {
          handleStockChange(data[0].id);
        }
      })
      .catch(err => setError(err.message));
  }, []);

  const handleStockChange = async (id) => {
    if (id === "upload") {
      setShowUpload(true);
      return;
    }
  }
}

```

```

    }

    setSelectedStock(id);
    setLoading(true);
    setError(null);

    try {
      const res = await fetch(`/api/fama-analysis/${id}`);
      const data = await res.json();
      setCurrentAnalysis(data);
    } catch (err) {
      setError(err.message);
    }

    setLoading(false);
  };

const handleUploadSuccess = (data) => {
  setCurrentAnalysis(data);
  setSelectedStock("");
  setShowUpload(false);
};

return React.createElement("div", { className: "bg-slate-950 min-h-screen text-white" },
  React.createElement(HeroSection),
  React.createElement("div", { id: "main-dashboard", className: "min-h-screen px-4 md:px-8 py-6 max-w-7xl mx-auto flex flex-col t

    React.createElement(Header),
    React.createElement(QuickGuide),

    error && React.createElement("div", { className: "bg-red-500/20 p-3 rounded mb-4 text-sm border border-red-500/30 text-red-200" },

    React.createElement(StockSelector, {
      stocks,
      selectedStock,
      onSelect: handleStockChange,
      onUploadClick: () => setShowUpload(true)
    )),

    loading ? React.createElement("div", { className: "flex-1 flex flex-col items-center justify-center py-20 gap-4" },
      React.createElement("svg", { className: "w-10 h-10 text-indigo-500 animate-spin", fill: "none", viewBox: "0 0 24 24" },
        React.createElement("circle", { className: "opacity-25", cx: "12", cy: "12", r: "10", stroke: "currentColor", strokeWidth: 2 },
          React.createElement("path", { className: "opacity-75", fill: "currentColor", d: "M4 12a8 8 0 018-8V0C5.373 0 0 5.373 0 12" }
        ),
        React.createElement("p", { className: "text-gray-400 font-semibold text-sm" }, "Running OLS Regression Analysis...")
      ) : currentAnalysis && React.createElement("div", { className: "space-y-6 animate-fade-in" },

      // Stock title banner card
      React.createElement("div", { className: "glass-panel p-5 border border-gray-800 flex flex-col sm:flex-row justify-between i
        React.createElement("div", {
          React.createElement("span", { className: "text-[10px] font-bold text-indigo-400 uppercase tracking-widest bg-indigo-500/50 p-2" },
            React.createElement("h2", { className: "text-2xl font-extrabold text-white mt-1.5 tracking-tight" }, currentAnalysis.stoc
          ),
          React.createElement(ExportReportButton, {
            stockName: currentAnalysis.stock_name,
            results: currentAnalysis.results
          })
        })
      ),

      // Beginner-friendly Card diagnostics
      React.createElement(ModelDiagnostics, { results: currentAnalysis.results }),

      // Grid Row: Results Table & Explanations side-by-side
      React.createElement("div", { className: "grid grid-cols-1 lg:grid-cols-3 gap-6" },
        React.createElement("div", { className: "lg:col-span-2 flex flex-col justify-between" },
          React.createElement(ResultsTable, { results: currentAnalysis.results }),

          React.createElement("div", { className: "glass-panel p-6 border border-gray-800/80 shadow-xl flex-1 flex flex-col justify-between" },
            React.createElement("h3", { className: "text-sm font-semibold text-gray-200 uppercase tracking-wider mb-4 flex items-center" },
              React.createElement("span", { className: "w-1.5 h-3 bg-indigo-500 rounded-full" }),
              " Stock Excess Return Trend (Ri - Rf)"
            ),
            React.createElement(ExcessReturnTrendChart, { rawData: currentAnalysis.data })
          )
        )
      )
    )
  )
);

```

```

    )
  ),
  React.createElement("div", {},
    React.createElement(InterpretationPanel, { results: currentAnalysis.results })
  )
),
// Grid Row: Bar exposure charts side-by-side (No blank columns!)
React.createElement("div", { className: "grid grid-cols-1 md:grid-cols-2 gap-6" },
  React.createElement("div", { className: "glass-panel p-6 border border-gray-800/80 shadow-xl" },
    React.createElement("h3", { className: "text-sm font-semibold text-gray-200 uppercase tracking-wider mb-4 flex items-center" },
      React.createElement("span", { className: "w-1.5 h-3 bg-indigo-500 rounded-full" }),
      " Factor Exposure Strength (Beta Loading)"
    ),
    React.createElement(FactorExposureChart, { factors: currentAnalysis.results.factors })
  ),
  React.createElement("div", { className: "glass-panel p-6 border border-gray-800/80 shadow-xl" },
    React.createElement("h3", { className: "text-sm font-semibold text-gray-200 uppercase tracking-wider mb-2 flex items-center" },
      React.createElement("span", { className: "w-1.5 h-3 bg-indigo-500 rounded-full" }),
      " Factor Significance (p-Value)"
    ),
    React.createElement("p", { className: "text-[11px] text-gray-500 mb-4" }, "Values below the red dashed line (p = 0.05) are significant")
    React.createElement(PValueComparisonChart, { factors: currentAnalysis.results.factors })
  )
),
),
React.createElement("footer", { className: "mt-12 py-6 border-t border-gray-900 text-center text-xs text-gray-600" },
  React.createElement("p", {}, "© 2026 FAMA AI Platform. Built for MBA + BTech Academic Research & Presentation.")
),
showUpload && React.createElement(UploadComponent, {
  onUploadSuccess: handleUploadSuccess,
  onClose: () => setShowUpload(false)
})
)
);
}

// =====
// SAFE RENDER (FIXED)
// =====
const rootEl = document.getElementById("root");

if (rootEl) {
  const root = ReactDOM.createRoot(rootEl);
  root.render(React.createElement(App));
}

```


■ Source File: backend/static/js/components.js

Language: JAVASCRIPT | Path: c:\Users\Sairal\Desktop\AntiGravity-Allbackend\static\js\components.js

```
const React = window.React;
const { useState, useEffect } = React;

// SVG Icons
const Icons = {
  Check: ({ className = "w-5 h-5" }) => (
    React.createElement("svg", { className, fill: "none", viewBox: "0 0 24 24", stroke: "currentColor", strokeWidth: "2" },
      React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round", d: "M5 13l4 4L19 7" })
    )
  ),
  Alert: ({ className = "w-5 h-5" }) => (
    React.createElement("svg", { className, fill: "none", viewBox: "0 0 24 24", stroke: "currentColor", strokeWidth: "2" },
      React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round", d: "M12 9v2m0 4h.01m-6.938 4h13.856c1.54 0 2.55-.596 3.13-1.718L20 7.718" })
    )
  ),
  Download: ({ className = "w-5 h-5" }) => (
    React.createElement("svg", { className, fill: "none", viewBox: "0 0 24 24", stroke: "currentColor", strokeWidth: "2" },
      React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round", d: "M4 16v1a3 3 0 003 3h10a3 3 0 003-3v-1m-4-4h.01m-3 0h5" })
    )
  ),
  Upload: ({ className = "w-5 h-5" }) => (
    React.createElement("svg", { className, fill: "none", viewBox: "0 0 24 24", stroke: "currentColor", strokeWidth: "2" },
      React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round", d: "M7 16a4 4 0 01-.88-7.903A5 5 0 115.9 6L16 6" })
    )
  ),
  Database: ({ className = "w-5 h-5" }) => (
    React.createElement("svg", { className, fill: "none", viewBox: "0 0 24 24", stroke: "currentColor", strokeWidth: "2" },
      React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round", d: "M4 7v10c0 2.21 3.582 4 8 4s8-1.79 8-4V7" })
    )
  ),
  Trend: ({ className = "w-5 h-5" }) => (
    React.createElement("svg", { className, fill: "none", viewBox: "0 0 24 24", stroke: "currentColor", strokeWidth: "2" },
      React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round", d: "M13 7h8m0 0v8m0-8l-8 8-4-6 6" })
    )
  ),
  Info: ({ className = "w-4 h-4" }) => (
    React.createElement("svg", { className, fill: "none", viewBox: "0 0 24 24", stroke: "currentColor", strokeWidth: "2" },
      React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round", d: "M13 16h-1v-4h-1m1-4h.01M21 12a9 9 0 11-18 0" })
    )
  ),
  Loader: ({ className = "w-5 h-5 animate-spin" }) => (
    React.createElement("svg", { className, fill: "none", viewBox: "0 0 24 24" },
      React.createElement("circle", { className: "opacity-25", cx: "12", cy: "12", r: "10", stroke: "currentColor", strokeWidth: "4" }),
      React.createElement("path", { className: "opacity-75", fill: "currentColor", d: "M4 12a8 8 0 018-8V0C5.373 0 4 5.373 4 12h4zm2 8" })
    )
  ),
  Academic: ({ className = "w-5 h-5" }) => (
    React.createElement("svg", { className, fill: "none", viewBox: "0 0 24 24", stroke: "currentColor", strokeWidth: "2" },
      React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round", d: "M12 6.253v13m0-13C10.832 5.477 9.246 5 7.5 5" })
    )
  )
};

// Tooltip helper component
function HelpTooltip({ text }) {
  const [visible, setVisible] = useState(false);
  return React.createElement("span", { className: "relative inline-block ml-1 cursor-help group text-gray-500 hover:text-indigo-400",
    React.createElement("span", {
      onMouseEnter: () => setVisible(true),
      onMouseLeave: () => setVisible(false),
      className: "inline-flex items-center justify-center w-3.5 h-3.5 text-[10px] font-bold border border-gray-500 rounded-full"
    }, "?"),
    visible && React.createElement("span", {
      className: "absolute bottom-full left-1/2 -translate-x-1/2 mb-2 w-48 p-2 text-xs bg-slate-900 border border-gray-800 text-gray-100"
    }, text)
  );
};
```

```

// 1. Header Component
export function Header() {
  return React.createElement("header", { className: "glass-panel px-6 py-4 flex items-center justify-between shadow-xl mb-6 border-l-1 border-r-1 border-t-1 border-b-1 border-gray-900/50" },
    React.createElement("div", { className: "flex items-center gap-3" },
      React.createElement("div", { className: "bg-gradient-to-tr from-indigo-500 to-cyan-400 p-2.5 rounded-lg shadow-lg shadow-indigo-500" },
        React.createElement(Icons.Academic, { className: "w-6 h-6 text-slate-900" })
      ),
      React.createElement("div", {},
        React.createElement("h1", { className: "text-xl font-bold tracking-tight text-white flex items-center gap-2" },
          "FAMA AI ",
          React.createElement("span", { className: "text-xs bg-indigo-500/20 text-indigo-400 px-2 py-0.5 rounded font-mono font-medium" }, "v1.0.0")
        ),
        React.createElement("p", { className: "text-xs text-gray-400" }, "Fama-French 3-Factor Stock Analysis Platform")
      )
    ),
    React.createElement("div", { className: "flex items-center gap-4" },
      React.createElement("div", { className: "hidden sm:flex items-center gap-2 px-3 py-1.5 rounded-full bg-slate-900/50 border border-gray-900/50" },
        React.createElement("span", { className: "w-2 h-2 rounded-full bg-emerald-500 animate-pulse" }),
        React.createElement("span", { className: "text-gray-400 font-medium font-mono" }, "OLS Engine: Active")
      )
    )
  );
};

// 1.5 Quick Start Tutorial Guide (Toggleable)
export function QuickGuide() {
  const [open, setOpen] = useState(false);

  return React.createElement("div", { className: "glass-panel p-4 border border-gray-800/80 mb-6 shadow-xl animate-fade-in card-grayscale" },
    React.createElement("div", { className: "flex justify-between items-center cursor-pointer select-none", onClick: () => setOpen(!open) },
      React.createElement("h4", { className: "text-xs font-bold text-indigo-400 uppercase tracking-widest flex items-center gap-2" },
        "Quick Start Guide",
        React.createElement("span", { className: "w-2 h-2 rounded-full bg-indigo-500 animate-pulse" })
      ),
      React.createElement("span", { className: "text-xs text-gray-500 font-bold font-mono" }, open ? "Collapse [-]" : "Expand [+]")
    ),
    open && React.createElement("div", { className: "grid grid-cols-1 md:grid-cols-3 gap-4 mt-4 border-t border-gray-900 pt-4 text-white" },
      React.createElement("div", { className: "p-3.5 bg-slate-950/40 rounded-xl border border-gray-900" },
        React.createElement("div", { className: "font-semibold text-gray-200 mb-1 flex items-center gap-1.5" },
          React.createElement("span", { className: "w-4.5 h-4.5 bg-indigo-500/20 text-indigo-400 rounded-full flex items-center justify-center" }, "Select target stock")
        ),
        React.createElement("p", { className: "text-gray-400 leading-relaxed font-normal" },
          "Choose pre-loaded NIFTY 50 data (Infosys, Tata Motors) or click the upload button to parse your custom factor dataset CSV"
        )
      ),
      React.createElement("div", { className: "p-3.5 bg-slate-950/40 rounded-xl border border-gray-900" },
        React.createElement("div", { className: "font-semibold text-gray-200 mb-1 flex items-center gap-1.5" },
          React.createElement("span", { className: "w-4.5 h-4.5 bg-indigo-500/20 text-indigo-400 rounded-full flex items-center justify-center" }, "Analyze factor tags")
        ),
        React.createElement("p", { className: "text-gray-400 leading-relaxed font-normal" },
          "Look at the top horizontal cards to instantly understand if the stock behaves as defensive/aggressive, growth/value, or momentum"
        )
      ),
      React.createElement("div", { className: "p-3.5 bg-slate-950/40 rounded-xl border border-gray-900" },
        React.createElement("div", { className: "font-semibold text-gray-200 mb-1 flex items-center gap-1.5" },
          React.createElement("span", { className: "w-4.5 h-4.5 bg-indigo-500/20 text-indigo-400 rounded-full flex items-center justify-center" }, "Hover over (?) symbols")
        ),
        React.createElement("p", { className: "text-gray-400 leading-relaxed font-normal" },
          "Confused by terms like R-squared, t-Stat, or p-Value? Hover your cursor over any question mark for a simple definition."
        )
      )
    )
  );
};

// 1.8 Hero Landing Page Component
export function HeroSection() {
  const [text, setText] = useState("");
  const [wordIndex, setWordIndex] = useState(0);
  const [isDeleting, setIsDeleting] = useState(false);
  const words = [
    "Fama-French Smart Analytics Platform",
  ]
}

```

```

    "Automated 3-Factor OLS Regression Engine",
    "Vibrant Plotly Financial Visualizations",
    "MBA + BTech Academic Project Excellence"
  ];

  const [animIndex, setAnimIndex] = useState(0);
  const [animKey, setAnimKey] = useState(0);

  useEffect(() => {
    const sequence = [
      { text: "F", isFull: false, duration: 250 },
      { text: "A", isFull: false, duration: 250 },
      { text: "M", isFull: false, duration: 250 },
      { text: "A", isFull: false, duration: 250 },
      { text: "A", isFull: false, duration: 250 },
      { text: "A", isFull: false, duration: 250 },
      { text: "I", isFull: false, duration: 250 },
      { text: "FAMA AI", isFull: true, duration: 1800 }
    ];

    const currentStep = sequence[animIndex];
    const timer = setTimeout(() => {
      setAnimIndex((prev) => (prev + 1) % sequence.length);
      setAnimKey((prev) => prev + 1);
    }, currentStep.duration);

    return () => clearTimeout(timer);
  }, [animIndex]);

  useEffect(() => {
    let timer;
    const currentWord = words[wordIndex];

    const tick = () => {
      if (!isDeleting) {
        setText(currentWord.substring(0, text.length + 1));
        if (text.length + 1 === currentWord.length) {
          timer = setTimeout(() => setIsDeleting(true), 2500);
          return;
        }
      } else {
        setText(currentWord.substring(0, text.length - 1));
        if (text.length - 1 === 0) {
          setIsDeleting(false);
          setWordIndex((wordIndex + 1) % words.length);
          timer = setTimeout(() => {}, 500);
          return;
        }
      }
    };

    const speed = isDeleting ? 30 : 60;
    timer = setTimeout(tick, speed);
  });

  timer = setTimeout(tick, 100);
  return () => clearTimeout(timer);
}, [text, isDeleting, wordIndex]);

const handleScrollDown = () => {
  const mainSection = document.getElementById("main-dashboard");
  if (mainSection) {
    mainSection.scrollIntoView({ behavior: "smooth" });
  }
};

return React.createElement("div", { className: "relative min-h-screen flex flex-col justify-center items-center px-4 overflow-hidden" },
  React.createElement("div", { className: "bg-gradient-orb" }),
  React.createElement("div", { className: "relative z-10 text-center max-w-4xl flex flex-col items-center gap-6" },
    React.createElement("div", { className: "bg-indigo-500/10 border border-indigo-500/20 text-indigo-400 text-[10px] font-bold" },
      "Academic equity research platform"
    ),
    React.createElement("div", { className: "title-container-fixed mb-2" },
      React.createElement("h1", {
        key: animKey,
        className: animIndex === 6
          ? "text-6xl sm:text-8xl font-black tracking-wider word-glow-in"

```

```

      : "text-7xl sm:text-9xl font-black letter-zoom-fade text-white drop-shadow-[0_0_20px_rgba(99,102,241,0.35)]"
    },
    animIndex === 6 ? "FAMA AI" : ["F", "A", "M", "A", "A", "I"][animIndex]
  )
),
React.createElement("div", { className: "h-16 flex items-center justify-center" },
  React.createElement("h2", { className: "text-lg sm:text-2xl font-medium text-gray-300 font-mono tracking-tight" },
    text,
    React.createElement("span", { className: "cursor-blink text-indigo-400 ml-1 font-sans font-bold" }, "|")
  )
),
React.createElement("p", { className: "text-xs sm:text-sm text-gray-500 max-w-xl leading-relaxed font-normal animate-slide-up" },
  "Analyze NIFTY 50 equities using the Fama-French 3-Factor pricing model. Built using FastAPI, Statsmodels OLS, React, and P
),
React.createElement("div", { className: "mt-12 animate-slide-up" },
  React.createElement("button", {
    onClick: handleScrollDown,
    className: "bg-gradient-to-r from-indigo-600 to-indigo-700 hover:from-indigo-500 hover:to-indigo-600 text-white font-semibol
  },
  "Launch Analytics Dashboard",
  React.createElement("svg", { className: "w-4 h-4", fill: "none", viewBox: "0 0 24 24", stroke: "currentColor", strokeWidth: 2 },
    React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round", d: "M14 5l7 7m0 0l-7 7m7-7H3" })
  )
),
),
),
React.createElement("div", { className: "absolute bottom-8 left-1/2 -translate-x-1/2 cursor-pointer flex flex-col items-center gap-2" },
  React.createElement("span", { className: "text-[10px] font-bold uppercase tracking-widest font-mono" }, "Scroll Down"),
  React.createElement("svg", { className: "w-5 h-5", fill: "none", viewBox: "0 0 24 24", stroke: "currentColor", strokeWidth: 2 },
    React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round", d: "M19 14l-7 7m0 0l-7 7m7-7V3" })
  )
),
);
}

// 2. Stock Selection Panel
export function StockSelector({ stocks, selectedStock, onSelect, onUploadClick }) {
  return React.createElement("div", { className: "glass-panel p-6 shadow-xl border border-gray-800/80 mb-6 animate-slide-up" },
    React.createElement("h3", { className: "text-sm font-semibold text-gray-400 uppercase tracking-wider mb-4 flex items-center gap-2" },
      React.createElement(Icons.Database, { className: "w-4 h-4 text-indigo-400" }),
      " Select Analysis Target"
    ),
    React.createElement("div", { className: "flex flex-col sm:flex-row gap-4 items-stretch sm:items-center" },
      React.createElement("div", { className: "flex-1 relative" },
        React.createElement("select", {
          value: selectedStock || "",
          onChange: (e) => onSelect(e.target.value),
          className: "w-full bg-slate-950 text-gray-200 border border-gray-800 rounded-lg py-2.5 px-4 focus:ring-2 focus:ring-indigo-400"
        },
          React.createElement("option", { value: "", disabled: true }, "-- Select a NIFTY 50 Stock --"),
          stocks.map((stock) => React.createElement("option", { key: stock.id, value: stock.id }, stock.name)),
          React.createElement("option", { value: "upload" }, "Upload Custom Factor Dataset (.csv)")
        )
      ),
      React.createElement("button", {
        onClick: onUploadClick,
        className: "flex items-center justify-center gap-2 bg-gradient-to-r from-indigo-600 to-indigo-700 hover:from-indigo-500 hover:to-indigo-600 text-white font-medium rounded-lg px-4"
      },
        React.createElement(Icons.Upload, { className: "w-4 h-4" }),
        " Upload Custom CSV"
      )
    )
  );
};

// 3. CSV Uploader Component
export function UploadComponent({ onUploadSuccess, onClose }) {
  const [dragActive, setDragActive] = useState(false);
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(false);

  const handleDrag = (e) => {
    e.preventDefault();
    e.stopPropagation();
  };

```

```

    if (e.type === "dragenter" || e.type === "dragover") {
      setDragActive(true);
    } else if (e.type === "dragleave") {
      setDragActive(false);
    }
  };

  const processFile = async (file) => {
    if (!file) return;
    if (!file.name.endsWith(".csv")) {
      setError("Only CSV files are accepted.");
      return;
    }
    setLoading(true);
    setError(null);

    const formData = new FormData();
    formData.append("file", file);

    try {
      const response = await fetch("/api/fama-analysis/upload", {
        method: "POST",
        body: formData,
      });

      if (!response.ok) {
        const errData = await response.json();
        throw new Error(errData.detail || "Error analyzing uploaded file.");
      }

      const data = await response.json();
      onUploadSuccess(data);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };

  const handleDrop = (e) => {
    e.preventDefault();
    e.stopPropagation();
    setDragActive(false);
    if (e.dataTransfer.files && e.dataTransfer.files[0]) {
      processFile(e.dataTransfer.files[0]);
    }
  };

  const handleFileChange = (e) => {
    if (e.target.files && e.target.files[0]) {
      processFile(e.target.files[0]);
    }
  };

  return React.createElement("div", { className: "fixed inset-0 bg-slate-950/80 backdrop-blur-sm flex items-center justify-center p-4" },
    React.createElement("div", { className: "glass-panel w-full max-w-lg overflow-hidden border border-gray-800 shadow-2xl animate-fade-in" },
      React.createElement("div", { className: "px-6 py-4 bg-slate-900 border-b border-gray-800 flex items-center justify-between" },
        React.createElement("h3", { className: "text-base font-bold text-white flex items-center gap-2" },
          React.createElement(Icons.Upload, { className: "w-5 h-5 text-indigo-400" }),
          " Upload Factor Dataset"
        ),
        React.createElement("button", {
          onClick: onClose,
          className: "text-gray-400 hover:text-white transition-colors p-1 hover:bg-slate-800 rounded"
        })
      ),
      React.createElement("svg", { className: "w-5 h-5", fill: "none", viewBox: "0 0 24 24", stroke: "currentColor" },
        React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round", strokeWidth: "2", d: "M6 18L18 6M6 6L18 18" })
      ),
      React.createElement("div", { className: "p-6" },
        React.createElement("div", {
          onDragEnter: handleDrag,
          onDragOver: handleDrag,
          onDragLeave: handleDrag,

```

```

      onDrop: handleDrop,
      className: `border-2 border-dashed rounded-xl p-8 flex flex-col items-center justify-center text-center cursor-pointer tra
      dragActive
        ? "border-indigo-500 bg-indigo-500/5"
        : "border-gray-800 bg-slate-950/30 hover:border-gray-700 hover:bg-slate-950/50"
    },
    onClick: () => document.getElementById("file-upload").click()
  },
  React.createElement("input", {
    id: "file-upload",
    type: "file",
    accept: ".csv",
    className: "hidden",
    onChange: handleFileChange,
    disabled: loading
  }),
  loading ? React.createElement("div", { className: "flex flex-col items-center gap-3 py-4" },
    React.createElement(Icons.Loader, { className: "w-10 h-10 text-indigo-400" }),
    React.createElement("p", { className: "text-sm text-indigo-300 font-medium" }, "Running OLS Regression Analysis...")
  ) : React.createElement(React.Fragment, {},
    React.createElement("div", { className: "bg-indigo-500/10 p-4 rounded-full mb-4 text-indigo-400" },
      React.createElement(Icons.Upload, { className: "w-8 h-8" })
    ),
    React.createElement("h4", { className: "text-sm font-semibold text-gray-200 mb-1" }, "Drag and drop your CSV file here"),
    React.createElement("p", { className: "text-xs text-gray-400 mb-4" }, "or click to browse your local machine"),
    React.createElement("div", { className: "bg-slate-900 border border-gray-800 px-3 py-1.5 rounded text-[11px] text-gray-400" },
      "Expected Columns: Date, Ri_Rf, Rm_Rf, SMB, HML"
    )
  ),
  ),
  error && React.createElement("div", { className: "mt-4 p-3.5 bg-rose-500/10 border border-rose-500/20 text-rose-300 rounded" },
    React.createElement("div", { className: "text-rose-400 mt-0.5" }, React.createElement(Icons.Alert, {})),
    React.createElement("div", {},
      React.createElement("p", { className: "font-semibold mb-0.5" }, "Upload Failed"),
      React.createElement("p", { className: "leading-relaxed" }, error)
    )
  ),
  React.createElement("div", { className: "mt-6 flex justify-end gap-3" },
    React.createElement("button", {
      onClick: onClose,
      className: "px-4 py-2 border border-gray-800 hover:bg-slate-900 hover:border-gray-700 text-gray-300 font-semibold rounded"
    }, "Cancel")
  )
)
)
);
}

// 4. Beginner-Friendly Diagnostics (Metrics Card Strip)
export function ModelDiagnostics({ results }) {
  if (!results) return null;
  const { r_squared, factors } = results;

  const mktBeta = factors.Market.coefficient;
  const smbBeta = factors.SMB.coefficient;
  const hmlBeta = factors.HML.coefficient;

  // Derive simple badges
  const getMktBadge = (val) => {
    if (val > 1.1) return { text: "Aggressive Risk", style: "text-rose-400 bg-rose-500/10 border-rose-500/20" };
    if (val < 0.9) return { text: "Defensive / Low Risk", style: "text-emerald-400 bg-emerald-500/10 border-emerald-500/20" };
    return { text: "Market Risk Match", style: "text-indigo-400 bg-indigo-500/10 border-indigo-500/20" };
  };

  const getCapBadge = (val) => {
    if (val > 0.15) return { text: "Small-Cap Stock", style: "text-orange-400 bg-orange-500/10 border-orange-500/20" };
    if (val < -0.15) return { text: "Large-Cap Giant", style: "text-blue-400 bg-blue-500/10 border-blue-500/20" };
    return { text: "Size Neutral", style: "text-gray-400 bg-gray-500/10 border-gray-800" };
  };

  const getStyleBadge = (val) => {
    if (val > 0.15) return { text: "Value stock (Undervalued)", style: "text-amber-400 bg-amber-500/10 border-amber-500/20" };
    if (val < -0.15) return { text: "Growth stock (Premium)", style: "text-fuchsia-400 bg-fuchsia-500/10 border-fuchsia-500/20" };
    return { text: "Blend Stock Style", style: "text-gray-400 bg-gray-500/10 border-gray-800" };
  };
}

```

```

const getReliability = (val) => {
  if (val > 0.85) return { text: "Highly Reliable", style: "text-emerald-400 bg-emerald-500/10 border-emerald-500/20" };
  if (val >= 0.50) return { text: "Moderate Fit", style: "text-amber-400 bg-amber-500/10 border-amber-500/20" };
  return { text: "Low Explanatory Power", style: "text-rose-400 bg-rose-500/10 border-rose-500/20" };
};

const mkt = getMktBadge(mktBeta);
const cap = getCapBadge(smbBeta);
const style = getStyleBadge(hmlBeta);
const rel = getReliability(r_squared);

return React.createElement("div", { className: "grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4 mb-6 animate-slide-up" },
  // Card 1: Market Risk
  React.createElement("div", { className: "glass-panel p-4 border border-gray-800/80 flex flex-col justify-between" },
    React.createElement("div", {},
      React.createElement("div", { className: "flex justify-between items-center mb-1" },
        React.createElement("span", { className: "text-xs font-semibold text-gray-400 uppercase tracking-wider" }, "Market Beta"),
        React.createElement(HelpTooltip, { text: "Measures sensitivity to overall market index (NIFTY 50). Beta > 1 moves more than the market" })
      ),
      React.createElement("h4", { className: "text-xl font-bold text-white font-mono" }, mktBeta.toFixed(2))
    ),
    React.createElement("div", { className: "mt-2" },
      React.createElement("span", { className: `text-[10px] font-bold px-2 py-0.5 rounded border ${mkt.style}` }, mkt.text)
    )
  ),
  // Card 2: Company Size
  React.createElement("div", { className: "glass-panel p-4 border border-gray-800/80 flex flex-col justify-between" },
    React.createElement("div", {},
      React.createElement("div", { className: "flex justify-between items-center mb-1" },
        React.createElement("span", { className: "text-xs font-semibold text-gray-400 uppercase tracking-wider" }, "Size Exposure"),
        React.createElement(HelpTooltip, { text: "Positive SMB means the stock responds like a smaller-cap stock. Negative means the opposite." })
      ),
      React.createElement("h4", { className: "text-xl font-bold text-white font-mono" }, smbBeta.toFixed(2))
    ),
    React.createElement("div", { className: "mt-2" },
      React.createElement("span", { className: `text-[10px] font-bold px-2 py-0.5 rounded border ${cap.style}` }, cap.text)
    )
  ),
  // Card 3: Stock Style
  React.createElement("div", { className: "glass-panel p-4 border border-gray-800/80 flex flex-col justify-between" },
    React.createElement("div", {},
      React.createElement("div", { className: "flex justify-between items-center mb-1" },
        React.createElement("span", { className: "text-xs font-semibold text-gray-400 uppercase tracking-wider" }, "Value / Growth"),
        React.createElement(HelpTooltip, { text: "Positive HML means it is a value stock (cheap relative to book value). Negative means it is a growth stock." })
      ),
      React.createElement("h4", { className: "text-xl font-bold text-white font-mono" }, hmlBeta.toFixed(2))
    ),
    React.createElement("div", { className: "mt-2" },
      React.createElement("span", { className: `text-[10px] font-bold px-2 py-0.5 rounded border ${style.style}` }, style.text)
    )
  ),
  // Card 4: Model Accuracy
  React.createElement("div", { className: "glass-panel p-4 border border-gray-800/80 flex flex-col justify-between" },
    React.createElement("div", {},
      React.createElement("div", { className: "flex justify-between items-center mb-1" },
        React.createElement("span", { className: "text-xs font-semibold text-gray-400 uppercase tracking-wider" }, "Model Explanatory Power"),
        React.createElement(HelpTooltip, { text: "R-squared shows what percentage of the stock's return movements is explained by the model." })
      ),
      React.createElement("h4", { className: "text-xl font-bold text-white font-mono" }, `${(r_squared * 100).toFixed(1)}%`)
    ),
    React.createElement("div", { className: "mt-2" },
      React.createElement("span", { className: `text-[10px] font-bold px-2 py-0.5 rounded border ${rel.style}` }, rel.text)
    )
  )
);
}

// 5. OLS Coefficients Table (with Tooltips)
export function ResultsTable({ results }) {
  if (!results) return null;
  const { factors } = results;

  const getPValueStyle = (val) => {
    if (val < 0.05) return "text-emerald-400 font-semibold";
  }
}

```

```

    return "text-gray-400";
  };

  const getFactorLabel = (factorKey) => {
    return {
      "Alpha": "Alpha (Intercept)",
      "Market": "Market Factor (Rm - Rf)",
      "SMB": "Size Factor (SMB)",
      "HML": "Style Factor (HML)"
    }[factorKey] || factorKey;
  };

  return React.createElement("div", { className: "glass-panel overflow-hidden border border-gray-800/80 shadow-xl mb-6 animate-slide" },
    React.createElement("div", { className: "px-6 py-4 bg-slate-900/50 border-b border-gray-800 flex items-center justify-between" },
      React.createElement("h3", { className: "text-sm font-semibold text-gray-200 tracking-wide uppercase flex items-center gap-2" },
        React.createElement("span", { className: "w-1.5 h-3 bg-indigo-500 rounded-full" }),
        " Statistical Regression Results (OLS)"
      ),
    ),
    React.createElement("div", { className: "overflow-x-auto text-xs sm:text-sm" },
      React.createElement("table", { className: "w-full text-left border-collapse styled-table" },
        React.createElement("thead", {},
          React.createElement("tr", { className: "border-b border-gray-800 bg-slate-950/40 text-gray-400 text-[11px] font-semibold" },
            React.createElement("th", { className: "py-3.5 px-5" },
              "Factor Name",
              React.createElement(HelpTooltip, { text: "The systemic market factor tested in the model." })
            ),
            React.createElement("th", { className: "py-3.5 px-5 text-right" },
              "Beta Coefficient",
              React.createElement(HelpTooltip, { text: "The slope or loading coefficient. A positive coefficient indicates direct sensitivity." })
            ),
            React.createElement("th", { className: "py-3.5 px-5 text-right" },
              "t-Stat",
              React.createElement(HelpTooltip, { text: "The test statistic. Values below -2 or above +2 indicate that the coefficient is statistically significant." })
            ),
            React.createElement("th", { className: "py-3.5 px-5 text-right" },
              "p-Value",
              React.createElement(HelpTooltip, { text: "The statistical probability of finding this result by chance. p-value < 0.05 indicates significance." })
            ),
            React.createElement("th", { className: "py-3.5 px-5 text-center" },
              "95% Confidence Interval",
              React.createElement(HelpTooltip, { text: "The range in which the true factor loading lies with 95% mathematical confidence." })
            ),
            React.createElement("th", { className: "py-3.5 px-5 text-center" },
              "Significant?",
              React.createElement(HelpTooltip, { text: "Reliable? Yes if p-value is less than 0.05, meaning this factor strongly affects returns." })
            )
          ),
        React.createElement("tbody", { className: "divide-y divide-gray-800/50" },
          Object.keys(factors).map((key) => {
            const factor = factors[key];
            return React.createElement("tr", { key, className: "hover:bg-slate-900/25 transition-colors" },
              React.createElement("td", { className: "py-3.5 px-5 font-semibold text-gray-300" }, getFactorLabel(key)),
              React.createElement("td", { className: "py-3.5 px-5 text-right font-mono text-gray-100 font-semibold" }, factor.coefficient),
              React.createElement("td", { className: "py-3.5 px-5 text-right font-mono text-gray-300" }, factor.t_stat.toFixed(2)),
              React.createElement("td", { className: "py-3.5 px-5 text-right font-mono text-gray-300" }, factor.p_value),
              React.createElement("td", { className: "py-3.5 px-5 text-center font-mono text-gray-400" }, factor.conf_lower.toFixed(2) + " - " + factor.conf_upper.toFixed(2)),
              React.createElement("td", { className: "py-3.5 px-5 text-center" },
                factor.significant
                ? React.createElement("span", { className: "inline-flex items-center gap-1.5 px-2 py-0.5 rounded-full text-xs font-semibold" },
                    React.createElement("span", { className: "w-1.5 h-1.5 rounded-full bg-emerald-400" }),
                    " Reliable"
                )
                : React.createElement("span", { className: "inline-flex items-center gap-1.5 px-2 py-0.5 rounded-full text-xs font-semibold" },
                    React.createElement("span", { className: "w-1.5 h-1.5 rounded-full bg-gray-500" }),
                    " Unreliable"
                )
              )
            )
          })
        )
      )
    )
  );
};

```



```

}

```

```

// 6. Interpretation Panel Component

```

```

export function InterpretationPanel({ results }) {
  if (!results) return null;
  const { interpretations, factors } = results;

  const getStyleTag = (key) => {
    if (key === "market") {
      const val = factors.Market.coefficient;
      if (val > 1.1) return { text: "Aggressive", color: "bg-rose-500/10 text-rose-400 border-rose-500/20" };
      if (val < 0.9) return { text: "Defensive", color: "bg-emerald-500/10 text-emerald-400 border-emerald-500/20" };
      return { text: "Market Core", color: "bg-indigo-500/10 text-indigo-400 border-indigo-500/20" };
    }
    if (key === "smb") {
      const val = factors.SMB.coefficient;
      if (val > 0.15) return { text: "Small-Cap Profile", color: "bg-orange-500/10 text-orange-400 border-orange-500/20" };
      if (val < -0.15) return { text: "Mega-Cap Giant", color: "bg-blue-500/10 text-blue-400 border-blue-500/20" };
      return { text: "Size Neutral", color: "bg-gray-500/10 text-gray-400 border-gray-800" };
    }
    if (key === "hml") {
      const val = factors.HML.coefficient;
      if (val > 0.15) return { text: "Value Style", color: "bg-amber-500/10 text-amber-400 border-amber-500/20" };
      if (val < -0.15) return { text: "Growth Style", color: "bg-fuchsia-500/10 text-fuchsia-400 border-fuchsia-500/20" };
      return { text: "Blend Style", color: "bg-gray-500/10 text-gray-400 border-gray-800" };
    }
    return { text: "Insight", color: "bg-gray-500/10 text-gray-400 border-gray-800" };
  };

  const getTitle = (key) => {
    return {
      "market": "Market Sensitivity (Beta)",
      "smb": "Company Size Exposure (SMB)",
      "hml": "Investment Style (HML)",
      "alpha": "Abnormal Outperformance (Alpha)",
      "r_squared": "Model Reliability (R²)"
    }[key] || key;
  };

  return React.createElement("div", { className: "glass-panel p-6 border border-gray-800/80 shadow-xl mb-6 animate-slide-up" },
    React.createElement("div", { className: "flex items-center gap-2 border-b border-gray-800 pb-4 mb-4" },
      React.createElement("div", { className: "text-indigo-400 bg-indigo-500/10 p-2 rounded-lg animate-pulse" },
        React.createElement(Icons.Trend, {})
      ),
      React.createElement("div", {},
        React.createElement("h3", { className: "text-sm font-bold text-white uppercase tracking-wider" }, "Fama-French Investment Insights"),
        React.createElement("p", { className: "text-xs text-gray-400" }, "Simplified translation of OLS statistical numbers")
      )
    ),
    React.createElement("div", { className: "space-y-4" },
      Object.keys(interpretations).map((key) => {
        const tag = getStyleTag(key);
        return React.createElement("div", { key, className: "p-4 bg-slate-950/40 border border-gray-900 rounded-xl hover:border-gray-800" },
          React.createElement("div", { className: "flex flex-wrap items-center justify-between gap-2 mb-2" },
            React.createElement("span", { className: "text-xs font-semibold text-gray-300 tracking-wide" }, getTitle(key)),
            React.createElement("span", { className: "text-[10px] font-bold px-2 py-0.5 rounded border ${tag.color}" }, tag.text)
          ),
          React.createElement("p", { className: "text-xs sm:text-sm text-gray-400 leading-relaxed font-normal" }, interpretations[key])
        );
      })
    )
  );
}

```

```

// 7. Report Export Component

```

```

export function ExportReportButton({ stockName, results }) {
  const [exporting, setExporting] = useState(false);

```

```

  const handleExport = async () => {
    if (!results || exporting) return;
    setExporting(true);

```

```

    try {
      const response = await fetch("/api/reports/export", {
        method: "POST",

```

```
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
        stock_name: stockName,
        results: results,
      }),
    });

    if (!response.ok) {
      throw new Error("Failed to download PDF report");
    }

    const blob = await response.blob();
    const url = window.URL.createObjectURL(blob);
    const a = document.createElement("a");
    a.href = url;
    a.download = `Fama_French_Report_${stockName.replace(/\s+/g, "_")}.pdf`;
    document.body.appendChild(a);
    a.click();
    window.URL.revokeObjectURL(url);
    document.body.removeChild(a);
  } catch (err) {
    alert(`Export Failed: ${err.message}`);
  } finally {
    setExporting(false);
  }
};

return React.createElement("button", {
  onClick: handleExport,
  disabled: exporting,
  className: "flex items-center justify-center gap-2 bg-gradient-to-r from-emerald-600 to-emerald-700 hover:from-emerald-500 hover:",
},
exporting ? React.createElement(React.Fragment, {},
  React.createElement(Icons.Loader, { className: "w-4 h-4 text-white" }),
  " Creating PDF..."
) : React.createElement(React.Fragment, {},
  React.createElement(Icons.Download, { className: "w-4 h-4" }),
  " Download PDF Report"
)
);
}
```

■ Source File: verify_fama_analysis.py

Language: PYTHON | Path: c:\Users\Sairal\Desktop\AntiGravity-AI\verify_fama_analysis.py

```
import os
import sys

# Add backend directory to path so we can import modules
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), 'backend')))

from utils.csv_loader import load_csv
from services.regression_service import run_fama_french_regression
from utils.pdf_generator import generate_pdf_report

def main():
    print("=====")
    print("FAMA AI - Programmatic Verification Test Suite")
    print("=====")

    # 1. Test sample files existence
    infosys_path = os.path.join("backend", "sample_data", "infosys.csv")
    tata_path = os.path.join("backend", "sample_data", "tata_motors.csv")

    print(f"Checking sample datasets...")
    assert os.path.exists(infosys_path), "Error: infosys.csv is missing"
    assert os.path.exists(tata_path), "Error: tata_motors.csv is missing"
    print("OK: Sample CSV files verified.")

    # 2. Test CSV Loader
    print(f"\nTesting CSV Loader on {infosys_path}...")
    try:
        df = load_csv(infosys_path)
        print(f"OK: CSV loaded successfully. Dimensions: {df.shape}")
        print(f"Columns: {list(df.columns)}")
        print(df.head(2))
    except Exception as e:
        print(f"FAIL: CSV loader failed with error: {str(e)}")
        sys.exit(1)

    # 3. Test OLS Regression Engine
    print("\nTesting OLS Regression Engine...")
    try:
        results = run_fama_french_regression(df)
        print("OK: OLS regression completed successfully.")
        print(f"Observations: {results['num_observations']}")
        print(f"R-squared: {results['r_squared']:.4f}")

        # Check factors
        for factor, details in results["factors"].items():
            print(f"  - {factor}: Coefficient={details['coefficient']:.4f}, p-val={details['p_value']:.4f}, significant={details['significant']}")

        print("\nInterpretations Sample:")
        print(f"  - Market: {results['interpretations']['market']}")
        print(f"  - Size: {results['interpretations']['smb']}")
        print(f"  - Style: {results['interpretations']['hml']}")
    except Exception as e:
        print(f"FAIL: OLS regression failed with error: {str(e)}")
        sys.exit(1)

    # 4. Test PDF Generator
    print("\nTesting ReportLab PDF Generator...")
    try:
        pdf_buffer = generate_pdf_report("Infosys Limited (TEST)", results)
        pdf_bytes = pdf_buffer.getvalue()
        print(f"OK: PDF generated in-memory. Size: {len(pdf_bytes)} bytes.")
        assert len(pdf_bytes) > 1000, "Error: PDF file size is suspiciously small"
    except Exception as e:
        print(f"FAIL: PDF generator failed with error: {str(e)}")
        sys.exit(1)

    print("\n=====")
    print("ALL TESTS COMPLETED SUCCESSFULLY!")
```

```
    print("=====")
if __name__ == "__main__":
    main()
```